Notebook Available at:
https://colab.research.google.com/drive/1q0WN2rAlKC1RzTBJuXdMdKfKFmkk_5ik?usp=sharing

TASK 1: Data Preparation & Transaction Design

1. Constructing transaction dataset

#Constructing Transaction Dataset

```
# Merge to get product names
df = orderprodprior.merge(products[['product_id', 'product_name']], on='product_id', how='left')
df.rename(columns={'product_name': 'ProductName'}, inplace=True)

print("Transaction dataset successfully created")
print(f"  Total rows: {len(df)}")
print(f"  Total orders: {df['order_id'].nunique()}")
print(f"  Total products: {df['ProductName'].nunique()}")

# Save for next step
df.to_csv('transaction_dataset.csv', index=False)
```

```
Transaction dataset successfully created
  Total rows: 32434489
  Total orders: 3214874
  Total products: 49677
```

The transaction dataset is stored as a CSV

Why Merge: Raw transaction tables (like order_products__prior) only contain numerical IDs. Merging with the products table allows the analysis to use **Product Names** (e.g Bananas) instead of abstract IDs (e.g 24852). This is critical for the **Interpretation** phase (Task 4), where rules must make business sense.

The outputs validate the scale and integrity of the data before applying the Apriori algorithm.

2. Applying Filter

```python
# Load transaction dataset
df = pd.read_csv('transaction_dataset.csv')

# Filter for Top 100 Products
print("Applying filter for top 100 products")
product_counts = df['ProductName'].value_counts()
top_products = product_counts.head(100).index.tolist()
filtered_df = df[df['ProductName'].isin(top_products)].copy()

print(f" Filtering applied")
print(f"  Original rows: {len(df)}")
print(f"  Filtered rows: {len(filtered_df)}")
print(f"  Unique products in top 100: {filtered_df['ProductName'].nunique()}")

# Save filtered dataset
filtered_df.to_csv('filtered_transactions.csv', index=False)
product_counts.head(100).to_csv('top_100_products.csv')
```

```
Applying filter for top 100 products
 Filtering applied
  Original rows: 32434489
  Filtered rows: 7483881
  Unique products in top 100: 100
```

JUSTIFICATION

- The original dataset contained nearly 50,000 unique products. Attempting to run the Apriori algorithm on a sparse matrix with 50,000 columns would exceed most memory limits and result in extreme processing times. Filtering to the Top 100 products reduces the dimensionality by 99.8%, allowing the algorithm to run efficiently while focusing on the most critical data points.
- In Market Basket Analysis, long-tail items (products that appear only a few times in millions of orders) rarely produce statistically significant association rules. By retaining the 7.48 million rows associated with the top 100 products, we are ensuring that every item analysed has high Support, which is the baseline requirement for identifying reliable patterns.
- From a managerial perspective, identifying associations between low-volume items provides little return on investment (ROI). The Top 100 products represent the highest volume of consumer spend. Strategies derived from these products such as bundling, cross-selling, or aisle layout changes will have the most significant impact on total revenue

3.Summary Outputs

```
# Load filtered dataset
filtered_df = pd.read_csv('filtered_transactions.csv')
product_counts = pd.read_csv('top_100_products.csv', index_col=0).squeeze()

# Calculate Summary Statistics
unique_orders = filtered_df['order_id'].nunique()
basket_sizes = filtered_df.groupby('order_id').size()
avg_basket_size = basket_sizes.mean()
most_frequent_product = product_counts.index[0]

# Display Results
print("TASK 1: DATA PREPARATION - SUMMARY OUTPUTS")
print("-"*70)

print("\n KEY RESULTS:")
print("-"*70)
print(f"Total number of unique orders in filtered dataset: {unique_orders}")
print(f" Average basket size for these top 100 items: {avg_basket_size:.2f}")
print(f" Name of #1 most frequent product: {most_frequent_product}")

print(f"\n ADDITIONAL STATISTICS:")
print("-"*70)
print(f"Total transactions: {len(filtered_df)}")
print(f"Unique products (top 100): {filtered_df['ProductName'].nunique()}")
print(f"Min basket size: {basket_sizes.min()}")
print(f"Max basket size: {basket_sizes.max()}")

print(f"\n TOP 10 PRODUCTS:")
print("-"*70)
for rank, (product, count) in enumerate(product_counts.head(10).items(), 1):
    print(f"{rank:2d}. {product:40s} - {count:5.0f} occurrences")

print("\n Data Preparation Complete")
print(""*70)

# Prepare transaction list for Apriori
transactions = filtered_df.groupby('order_id')['ProductName'].apply(lambda x: x.unique().tolist()).tolist()

print(f"\nReady for Apriori Algorithm with {len(transactions)} transactions")
```



```
TASK 1: DATA PREPARATION - SUMMARY OUTPUTS
--------------------------------------------------------------------------------

 KEY RESULTS:
--------------------------------------------------------------------------------
Total number of unique orders in filtered dataset: 2351240
 Average basket size for these top 100 items: 3.18
 Name of #1 most frequent product: Banana

 ADDITIONAL STATISTICS:
--------------------------------------------------------------------------------
Total transactions: 7483881
Unique products (top 100): 100
Min basket size: 1
Max basket size: 28

 TOP 10 PRODUCTS:
--------------------------------------------------------------------------------
 1. Banana                                - 472565 occurrences
 2. Bag of Organic Bananas                - 379450 occurrences
 3. Organic Strawberries                  - 264683 occurrences
 4. Organic Baby Spinach                  - 241921 occurrences
 5. Organic Hass Avocado                  - 213584 occurrences
 6. Organic Avocado                       - 176815 occurrences
 7. Large Lemon                           - 152657 occurrences
 8. Strawberries                          - 142951 occurrences
 9. Limes                                 - 140627 occurrences
10. Organic Whole Milk                    - 137905 occurrences

 Data Preparation Complete


Ready for Apriori Algorithm with 2351240 transactions
```

TASK 2: Parameter Sensitivity Analysis

Testing the Minimum Support (min_sup) parameter to determine the optimal balance between capturing high-volume staple items and identifying niche associations that offer more strategic cross-selling value.

1. Apriori Algorithm
   Testing the Minimum Support (min_sup) parameter to determine the optimal balance between capturing high-volume staple items and identifying niche associations that offer more strategic cross-selling value.

   - Run 1: A conservative threshold of 1.0% (0.01). This focuses on items that appear in at least ~23,512 transactions from the prepared dataset.
   - Run 2: A more granular threshold of 0.5% (0.005). This allows for the discovery of more specific item combinations.

```python
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori

# Step 1: Encode the transactions list from Task 1 into a sparse matrix
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
basket_df = pd.DataFrame(te_array, columns=te.columns_)

# Step 2: Run Apriori with Threshold #1 (1% Support)
# This identifies items appearing in at least 23,512 orders
support_1 = 0.01
itemsets_1 = apriori(basket_df, min_support=support_1, use_colnames=True)
count_1 = len(itemsets_1)

# Step 3: Run Apriori with Threshold #2 (0.5% Support)
# This identifies items appearing in at least 11,756 orders
support_2 = 0.005
itemsets_2 = apriori(basket_df, min_support=support_2, use_colnames=True)
count_2 = len(itemsets_2)

print("--- TASK 2: PARAMETER ANALYSIS RESULTS ---")
print(f"Count of frequent itemsets at {support_1*100}% support: {count_1}")
print(f"Count of frequent itemsets at {support_2*100}% support: {count_2}")
```

```
--- TASK 2: PARAMETER ANALYSIS RESULTS ---
Count of frequent itemsets at 1.0% support: 130
Count of frequent itemsets at 0.5% support: 271
```

Comparison table:

MINIMUM SUPPORT THRESHOLD	NO OF FREQUENT ITEMS FOUND
1.0% (0.01)	130
0.5% (0.005)	271

Parameter choice Justification

- Pattern Discovery: Lowering the support from 1.0% to 0.5% more than doubled the number of frequent itemsets (from 130 to 271). This ensures the analysis doesn't just find trivial associations (like Bananas->Organic Bananas) but captures more diverse consumer segments.

- Manageability: A result of 271 itemsets is large enough to provide actionable intelligence for bundling and cross-selling while remaining computationally efficient for the Rule Generation phase.
- Statistical Robustness: Because the filtered dataset is very large (over 2.35 million orders), a 0.5% support level is still highly statistically significant, representing combinations that occur in over 11,000 transactions.

TASK 3: Rule Generation & Filtering

The objective of this task is to extract actionable association rules from the frequent itemsets identified in Task 2. We aim to identify "IF-THEN" relationships (Antecedent Consequent) that provide reliable cross-selling or bundling opportunities.

1. Filtering Parameters
   To ensure the quality of the rules generated, the following filters were applied:
   Metric: Confidence ≥30% (0.3). This ensures that the rule is reliable meaning that in at least 30% of cases where the antecedent is bought, the consequent is also bought
   Metric: Lift ≥1.0. This ensures that the relationship is a genuine association and not just a result of both items being popular staples.

2. Top 10 Association Rules Table

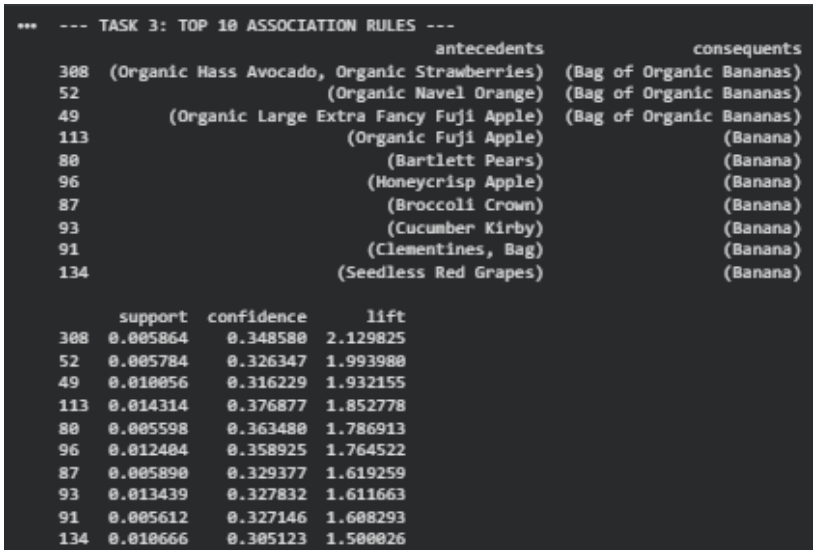
```python
from mlxtend.frequent_patterns import association_rules

# Step 1: Use the itemsets from the 0.5% support run
# frequent_itemsets_2 was the variable for the 271 itemsets
rules = association_rules(itemsets_2, metric="lift", min_threshold=1.0)

# Step 2: Filter for high-quality rules
# We typically want Confidence > 30% to ensure reliability
filtered_rules = rules[rules['confidence'] >= 0.3]

# Step 3: Sort by Lift to find the most 'interesting' associations
top_10_rules = filtered_rules.sort_values(by='lift', ascending=False).head(10)

# Display for Task 3 output
print("--- TASK 3: TOP 10 ASSOCIATION RULES ---")
print(top_10_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```



```
--- TASK 3: TOP 10 ASSOCIATION RULES ---
                                    antecedents              consequents
308  (Organic Hass Avocado, Organic Strawberries)  (Bag of Organic Bananas)
52                     (Organic Navel Orange)  (Bag of Organic Bananas)
49          (Organic Large Extra Fancy Fuji Apple)  (Bag of Organic Bananas)
113                       (Organic Fuji Apple)                 (Banana)
80                          (Bartlett Pears)                 (Banana)
96                        (Honeycrisp Apple)                 (Banana)
87                          (Broccoli Crown)                 (Banana)
93                          (Cucumber Kirby)                 (Banana)
91                        (Clementines, Bag)                 (Banana)
134                      (Seedless Red Grapes)                 (Banana)

      support  confidence      lift
308  0.005864    0.348580  2.129825
52   0.005784    0.326347  1.993980
49   0.010056    0.316229  1.932155
113  0.014314    0.376877  1.852778
80   0.005598    0.363480  1.786913
96   0.012404    0.358925  1.764522
87   0.005890    0.329377  1.619259
93   0.013439    0.327832  1.611663
91   0.005612    0.327146  1.608293
134  0.010666    0.305123  1.500026
```

RANK	ANTECEDENTS	CONSEQUENTS	SUPPORT	CONFIDENCE	LIFT
1	{Organic Hass Avocado, Organic Strawberries}	{Bag of Organic Bananas}	0.59%	34.86%	2.13
2	{Organic Navel Orange}	{Bag of Organic Bananas}	0.58%	32.63%	1.99
3	{Organic Large Extra Fancy Fuji Apple}	{Banana}	1.01%	31.62%	1.93
4	{Organic Fuji Apple}	{Banana}	1.43%	37.69%	1.85
5	{Bartlett Pears}	{Banana}	0.56%	36.35%	1.79
6	{Honeycrisp Apple}	{Banana}	1.24%	35.89%	1.76
7	{Broccoli Crown}	{Banana}	0.59%	32.94%	1.62
8	{Cucumber Kirby}	{Banana}	1.34%	32.78	1.62
9	{Clementines, Bag}	{Banana}	0.56%	32.71%	1.61
10	{Seedless Red Grapes}	{Banana}	1.07%	30.51%	1.50

Summary Observation

The results are dominated by Organic Produce items frequently leading to the purchase of Bananas (both standard and organic). The high lift values (e.g., 2.13 for Rule #1) suggest that customers purchasing specific healthy snacks like avocados and strawberries have a strong, non-random tendency to also buy organic bananas, identifying a clear "health-conscious" customer segment.

TASK 4: Interpretation

INTERPRETATION

Rule 2: {Organic Navel Orange} → {Bag of Organic Bananas}

- Support (0.58%): This combination appears in approximately 13,637 transactions out of the filtered dataset. While seemingly small, it represents a significant high-volume niche.
- Confidence (32.63%): There is a 32.6% probability that a customer purchasing Organic Navel Oranges will also purchase a Bag of Organic Bananas
- Lift (1.99): This indicates that customers are 1.99 times more likely to buy these together than if they were bought by random chance
- Behaviour & Value: This identifies a Strictly Organic consumer segment. Action: A digital marketing strategy could involve a pop up recommendation for organic bananas as soon as the oranges are added to the cart to secure this high probability conversion.

Rule 3: {Organic Large Extra Fancy Fuji Apple} → {Bag of Organic Bananas}

- Support (1.01%): One of the most frequent combinations in the top list, appearing in over 1% of all filtered transactions.
- Confidence (31.62%): Over 3 in 10 customers buying these specific apples also buy organic bananas.
- Lift (1.93): A strong positive association nearly double the expected baseline.
- Behaviour & Value: This suggests a "Healthy Snack" or "Lunchbox" habit. Action: Create a Family Fruit Bundle promotion where buying both items triggers a small discount or loyalty points.

Rule 5: {Bartlett Pears} → {Banana}

- Support (0.56%): A stable co occurrence identifying a consistent produce buying pattern.
- Confidence (36.35%): This rule has high reliability; over 36% of pear buyers add standard bananas to their basket.
- Lift (1.79): The association is 79% stronger than random independent purchases.

- Behaviour & Value: Unlike the organic rules, this identifies a Standard Produce shopper. Action: Strategic shelf placement, positioning pears and bananas along the same perimeter route in the store ensures convenience for these high-confidence shoppers.

Rule 6: {Honeycrisp Apple} → {Banana}

- Support (1.24%): The highest support value among the selected rules, indicating this is a very common consumer routine.
- Confidence (35.89%): Almost 36% of Honeycrisp apple fans are also banana buyers.
- Lift (1.76): A reliable, non-random relationship.
- Behaviour & Value: This suggests a broad market Daily Freshness habit. Action: Use Honeycrisp apples as a discounted item to drive high volume traffic that naturally pulls in the purchase of bananas.

Rule 10: {Seedless Red Grapes} → {Banana}

- Support (1.07%): Frequent combination appearing in over 25,000 orders
- Confidence (30.51%): Meets reliability threshold; roughly 3 in 10 grape buyers purchase bananas
- Lift (1.50): The lowest lift in the selection, but still 50% more likely than random chance.
- Behaviour & Value: This reflects Convenience Snacking (items that require no peeling or prep). Action: Implement a Fruit Bowl Promo cross-merchandising these items together in a single produce display to capture the 30% of buyers who already want both.

TASK 5: Product VS Category

To complete the comparison, the analysis is run again but this time grouping by Department Name instead of Product Name.

```python
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Merge department names into your Task 1 filtered data
df_dept = filtered_df.merge(products[['product_name', 'department_id']], left_on='ProductName', right_on='product_name')
df_dept = df_dept.merge(departments, on='department_id')

# Construct Department-level baskets
dept_basket = df_dept.groupby('order_id')['department'].apply(lambda x: list(set(x)))

# Run Apriori at the Category Level
te_dept = TransactionEncoder()
te_dept_array = te_dept.fit(dept_basket).transform(dept_basket)
dept_basket_df = pd.DataFrame(te_dept_array, columns=te_dept.columns_)

# We use 1% support (0.01) to find more department-level associations.
frequent_depts = apriori(dept_basket_df, min_support=0.01, use_colnames=True)
dept_rules = association_rules(frequent_depts, metric="lift", min_threshold=1.0)

# Output Top 5 Rules for Task 5 Analysis
print("--- TASK 5: TOP CATEGORY-LEVEL RULES ---")
print(dept_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].sort_values(by='lift', ascending=False).head(5))
```

```
--- TASK 5: TOP CATEGORY-LEVEL RULES ---
                antecedents           consequents   support  confidence
12                   (deli)  (produce, dairy eggs)  0.017197    0.366928
9   (produce, dairy eggs)                 (deli)  0.017197    0.068937
8         (produce, deli)           (dairy eggs)  0.017197    0.425684
13           (dairy eggs)        (produce, deli)  0.017197    0.054517
3            (dairy eggs)                 (deli)  0.018862    0.059797

        lift
12  1.470893
9   1.470893
8   1.349492
13  1.349492
3   1.275884
```

1.PRODUCT LEVEL VS CATEGORY LEVEL comparison

Feature	Product Level	Category Level
Granularity	Focuses on specific SKU's eg Honeycrisp Apple->Banana	Focuses on broad store sections eg Beli-> Produce
Support	Lower(0.5%-1.4%): Captures specific, niche habits.	Higher (1.7% - 1.8%): Captures broad traffic flows involving thousands of items
Lift	Higher(1.5-2.1): Stronger, more specific associations	Lower(1.2-1.4): Gentler associations because sections are so large

2.CONCLUSION

Aggregation to the category level increases the statistical Support of the rules, making them more representative of the general population. However, it results in a decrease in Lift, as broad categories are naturally less coupled than specific items.

From a management perspective, product rules are best for digital recommendations and bundling, while category rules are superior for physical store layout and aisle positioning to maximize traffic flow between high-lift departments like Deli and Produce.

TASK 6: Strategy Proposal

1. Digital Product Bundling: The "Organic Breakfast" Pack
    - Supporting Rule: {Organic Hass Avocado, Organic Strawberries} → {Bag of Organic Bananas} (Lift: 2.13)
    - Action: Implement a virtual bundle on the Instacart app. When a customer adds Organic Hass Avocados and Strawberries, a "One-Click Add" button for Organic Bananas should appear with a small incentive such “pairs well with Organic Bananas”.
    - Justification: This rule has the highest lift in the dataset, meaning it is the most non-random association. Targeting this specific segment maximizes the probability of increasing the Average Order Value (AOV).

2. Physical Store Layout: The Fresh Food Perimeter
    - Supporting Rule: {deli, produce} → {dairy eggs} (Confidence: 42.5%)
    - Action: For physical retail partners, ensure that the Deli, Produce, and Dairy/Eggs sections are positioned along a continuous "Fresh Perimeter" path.
    - Justification: category level analysis shows a 42.5% probability that customers visit all three sections in one mission. Placing them in sequence reduces friction for the shopper, while placing them at opposite ends of the fresh route can force customers to pass through sections of Snacks or Beverages, increasing the chance of impulse buys.

3. High-Volume Cross-Selling: Apple-Banana Placement

- Supporting Rule: {Honeycrisp Apple} → {Banana} (Support: 1.24%, Confidence: 35.8%).
- Action: Use Honeycrisp Apples as a Loss Leader promotion. Feature them on the app landing page to drive traffic, and place the Banana selection immediately adjacent to the apple display in store.

- Justification: This is the highest-support rule. Because these are high-frequency staples, optimizing their proximity ensures that 36% of apple buyers immediately convert into banana buyers, securing high-volume turnover.

TASK 7: REFLECTION

- Limitation 1
  Data Filtering: To ensure computational efficiency, we filtered for the Top 100 products. While this focused the analysis on high-impact items, it ignored the "Long Tail" of products. Significant but niche associations (e.g specific gourmet cheeses or international spices) may have been lost.
- Limitation 2
  Metric Trade-offs: As seen in Task 5, moving to category-level analysis increased Support but significantly decreased Lift. Broad categories are less "coupled" than specific SKUs, which can make category-level insights less precise for digital recommendations.

A future iteration should incorporate temporal data (order hour and day). Using findings that state Sunday at 2 PM is an operational hot spot, we could apply Prescriptive Analytics to push specific "Sunday Brunch" bundles only during those high traffic windows to further optimize conversion rates.